

Lane Detection using Graph Search and Geometric Constraints for Formula Student Driverless

Ivo Ivanov, Carsten Markgraf

Abstract—Lane detection is a fundamental task in autonomous driving. While the problem is typically formulated as the detection of continuous boundaries, we study the problem of detecting lane boundaries that are sparsely marked by 2D points with many false positives. This problem arises in the Formula Student Driverless (FSD) competition and is challenging due to its inherent ambiguity. Previous methods are inefficient and unable to find long-horizon solutions. We propose a deterministic algorithm called CLC that uses backtracking graph search with a learned likelihood function to overcome these limitations. We impose geometric constraints on the lane candidates to guarantee a geometrically sound lane. Our exhaustive search leads to finding the global optimum in 45% of instances, and the algorithm is overall robust to up to 50% false positives. Our algorithm runs in less than 15 ms on a single CPU core, meeting the low latency requirements of autonomous racing. We extensively evaluate our method on real data and realistic racetrack layouts, and show that it outperforms the state-of-the-art by detecting long lanes over 100 m with few (0.6%) critical failures. This allows our autonomous racecar to drive close to its physical limits on a previously unknown racetrack without being limited by perception. We release our dataset with realistic Formula Student racetracks to enable further research.

Index Terms— autonomous Racing, Lane Detection, Formula Student, Graph Search, Combinatorial optimization, Geometric Constraints

I. INTRODUCTION



Figure 1: The autonomous race car of the team StarkStrom Augsburg at the Formula Student Germany competition 2023 on the race track marked by blue and yellow traffic cones. (Copyright FSG Lodholz).

Autonomous vehicle racing is a challenging application of autonomous driving, featuring competitions like the Indy Autonomous Challenge, the F1TENTH competition, and the Formula Student Driverless (FSD) competition [1]. Autonomous racing poses numerous challenges to the system and algorithms employed. Due to the high speeds, a high degree of

Ivo Ivanov and Carsten Markgraf are with the Faculty of Electrical Engineering at the Technical University Augsburg, Augsburg, Germany. Corresponding author Email: ivo.ivanov@hs-augsburg.de

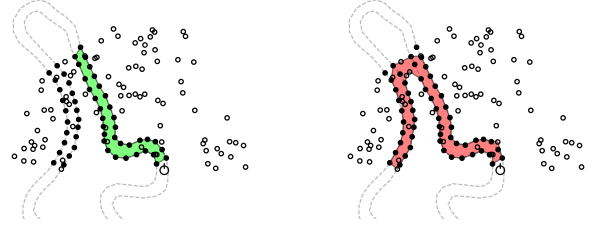


Figure 2: Detecting the driving lane (outlined in gray) on race tracks where the boundaries are marked only sparsely by points and with many false-positives (empty circles) introduces ambiguity: On the left is the correct prediction, but given only the partial observation of the boundary points, it may be detected incorrectly as shown on the right, taking a shortcut.

uncertainty originates from short perception intervals, at the same time a low-latency perception and processing is required. When driving at high speeds up to 20 m/s (72 km/h) on an unknown track in the FSD-competition, reliable lane detection is crucial for optimal planning of the racing line to achieve a minimum lap time.

This paper focuses on lane detection using only 2D-points, a problem arising in the FSD competition where the lane is marked with traffic cones (Fig. 1). Other potential applications of the approach presented in this paper besides the FSD competition may include detecting lanes on road construction sites [2] and making the lane detection more robust to unreliable perception as well as resolving ambiguity arising due to damaged lane markings.

The FSD competition is a student engineering competition on autonomous racing using self-developed race vehicles. Among several disciplines in this competition, the *utocross* is the most challenging one for the perception and planning systems as the track layout is not known in advance. In this discipline, the vehicle must drive one lap autonomously on a closed circuit in the minimum time without leaving the track.

As typical for autonomous robots, the perception pipeline of an FSD race car first detects cones with LiDAR-sensors and cameras and then builds a map of the environment with SLAM-algorithms using these detected cones, usually as a set of 2D points [3, 4, 5], a landmark map. After mapping, the objective is to detect the two boundaries in this set of points defining the lane.

Lane detection is a well-studied task in autonomous driving where continuous curves are detected in camera images or LiDAR point clouds [6, 7]. In the FSD competition however,

the problem is quite different as the boundaries are marked by points as opposed to continuous lines. Also, lanes of racetracks are usually much more curvy and also the high number of false-positives introduces ambiguities (see Fig. 2).

Previous methods are only able to detect short sections of the track, also they are prone to false-positives and inefficient. Proposed methods based on mixed-integer optimization are robust but do not scale well with growing number of input points due to long solve times and have thus only a short horizon [8]. The motivation of our work is therefore the question: *Can we detect reliably a driving lane sparsely marked by points and many false-positives over a long-horizon, enabling an FSD-race car to utilize its full performance?*

We propose a deterministic lane detection algorithm named Cone Lane Connector (CLC), which uses a modified depth-first graph search combined with geometric constraint checking to find candidate solutions. This step makes our approach not only robust by guaranteeing geometrically sound lanes, but also efficient by allowing pruning of the search space through backtracking. In a second step, we use a machine learning approach to find the most likely candidate lane by ranking. Due to the exhaustive search with effective heuristics, our method is able to cope with a high number of false positives. Surprisingly, although the combinatorial solution space of two paths in a graph is prohibitively large, enumeration becomes feasible when imposing these geometric constraints. This allows us to find the global optimum in many cases. The machine learning approach to ranking also eliminates the need for hand-tuning heuristics.

We summarize the contributions of this work as follows:

- 1) Using heuristically guided graph search that enumerates simultaneously two paths in a graph (path pairs) for finding feasible lanes with boundaries marked by points (Fig. 2)
- 2) Carefully designed geometric constraints which guarantee lane soundness and allow for an effective pruning of the search space with backtracking
- 3) Using a neural network for ranking to find the most likely lane
- 4) An open dataset with real formula student race-track layouts including annotated ground-truth driving lanes

We released our dataset with racetrack layouts at ¹. Video demonstrating the overall performance of our autonomous race car driving at speeds up to 20 m/s (72 km/h) on a previously unknown race track using the proposed lane detection is available at: <https://youtu.be/9MWKDJeEDU>

II. RELATED WORK

A. Lane detection from boundaries marked by points

Although lane detection is a well-studied topic in autonomous driving, only a few works address the problem of detecting lanes marked by points, most of them are directly related to the FSD-competition. In [4], Kabzan et al. describe an approach for lane detection for the FSD competition based on exhaustive search with breadth-first search (BFS) and

beam search. They construct the search graph via Delaunay triangulation and formulate the likelihood of candidate lanes using handcrafted geometric features. They do not use hard constraints, therefore geometrically sound solutions are only favored instead of enforced. This work is improved in [3] where Andersen et al. propose to use Bayesian inference approach for deriving the likelihood of a lane based on the color of all observed cones and hand-tuned probabilities. detection horizon of up to 15m is reported which is rather short and limits the planning horizon.

In [8], Brandes et al. also study lane detection for the FSD-competition by framing it as a constraint optimization problem in the domain of two paths in a graph. They formulate geometric constraints and use a handcrafted geometric likelihood function to select the most likely candidate. As a solving strategy, they formulate the problem as a constrained binary integer optimization problem to be able to use off-the-shelf solvers (e.g. *Mosek*). While their declarative approach is elegant, it suffers from inefficiency – it scales with the number of points in the map input instead of the number of feasible candidate solutions. This requires limiting the number of input points to only 12-16 in order to meet the real-time requirements, which makes the overall detection horizon rather short.

In [2], Tanzmeister et al. study lane detection of a road lane marked with traffic cones, which is the same problem as it arises in the FSD-competition. Instead of detecting the lane boundaries directly, a path inside the lane is detected. The authors use the path planning algorithms *Rapidly-exploring Random Trees* (RRT) and A^* as a search strategy and impose kinematic constraints. Although RRT seems to work for this lane detection problem where the boundaries are marked by points, path planning has the different objective of finding a continuous curve in a continuous search space [9, 10, 11]. The problem of detecting boundaries from a set of points on the other hand has a much smaller solution set. Thus, the solution set is oversampled, making the algorithm conceptually inefficient. In fact, as the authors note: "Many of the paths are similar and thus lead to the same road boundary".

B. Lane detection

More commonly, lane detection is performed by detecting continuously marked lane boundaries. Typical lane detection algorithms deal with computer vision aspects such as invariance to lighting and noise, classical approaches such as [6, 12, 13, 14] use camera images and first detect edges, then use Hough voting to estimate the most likely parameters for splines modeling the boundaries. From multiple solutions, the most likely one is selected by handcrafted heuristics. A different approach is proposed in [15] where the authors use a graph search in the image space to detect unstructured roads. They make use of geometric prior information such as the minimum and maximum lane width and combine it with other handcrafted heuristics.

Deep-learning based approaches such as [16, 17, 18] use neural networks for either boundary detection or segmentation of the lane area. Geometric properties known a-priori like

¹https://github.com/iv461/fsd_racetrack_dataset

straightness and small lane width variance have been used to improve lane detection in [19]. The idea of using geometric constraints to enforce geometrically sound lanes has been proposed in [20] and incorporated as a post-processing step instead for making the search more efficient. Using handcrafted geometric features based on the variance has been used [21] to select the most likely lane candidate via a binary classifier.

C. Path planning

Another related line of work are path planning algorithms used for mobile robots and autonomous vehicles. Path planning searches for a single path within a continuous search space (2D or 3D). This space may also be a traversable area, possibly with obstacles, and the task is to find a path from a starting point to an end point that avoids the obstacles while satisfying kinematic constraints [11]. In addition, the path planning algorithm usually optimizes for path length, curvature, etc. [22]. In [23], the authors propose an algorithm based on A^* for real-time path planning in an urban environment, using heuristics-guided graph search to find kinematically feasible solutions and a subsequent smoothing step. In autonomous racing, path planning is approached via trajectory optimization given an already detected lane [24].

D. Graph search techniques

For many combinatorial search and optimization problems with a prohibitively large solution set, specialized search techniques have been proposed. Search heuristics are used to find good locally optimal solutions, for this machine learning based methods using graph neural networks (GNNs) have been proposed in [25] to improve graph algorithms. Even for fundamental graph problems it has been shown that learned heuristics can outperform handcrafted heuristics [26].

III. OVERVIEW

Algorithm 1 CLC: Lane detection in a map of 2D points.

Input: $\mathcal{M}$: set of 2D points representing the map, pose_{car} : pose of the car in map coordinate system

Output: Most likely driving lane

```

1: function CLC( $\mathcal{M}$ ,  $\text{pose}_{car}$ )
2:    $\mathcal{G} \leftarrow \text{construct\_search\_graph}(\mathcal{M}, d_{max})$ 
3:    $s_l, s_r \leftarrow \text{find\_starting\_vertices}(\mathcal{M}, \text{pose}_{car})$ 
4:   % Enumerate path pairs representing lanes ( Alg. 2)
5:    $\text{candidates} \leftarrow \text{EPP}(\mathcal{G}, (s_l, s_r))$ 
6:   return  $\text{most\_likely\_candidate}(\text{candidates})$ 
```

A. Generative model of the map

The perception system of the autonomous vehicle first builds a map of the environment. This map is the input to the lane detection algorithm and is typically obtained in 2D, as the height deviation is negligible. We assume the following generative model of the map. The map $\mathcal{M}$ consists of measured points $p_1, p_2, \dots, p_N \in \mathbb{R}^2$ which are noisy. We assume that the points are perturbed by Gaussian noise. The

measurement error which is typically achieved in such maps produced by LiDAR sensors is 0.2m - 0.3m [3].

Additionally, there may be false-positive points, i.e. not every point in the map may actually be present in the environment. We define the false-positive rate (FP-Rate) as the fraction of false-positive points of the total number of points in the map. If the point is a false-positive, we assume it is sampled from a uniform distribution in the range of the perception field. We measured values from 10% to 30% for the false-positive rate of our perception pipeline, which are similar to the 15% - 25% reported in [8].

Every cone also has a color, indicating whether it belongs to the left or right boundary, but our algorithm does not use this information. We observed that in our perception pipeline, the color information is unreliable especially at large distances, and also wrong colors are strongly correlated, violating the independence assumption made in [3].

B. Problem statement and algorithm outline

Given a map of 2D points generated by the generative model, the problem of lane detection is to find two sequences of map points that represent the boundaries of the lane.

Additionally, the algorithm should only output geometrically sound lanes, where soundness is defined by a set of constraints.

As an additional input besides the map points, we use the pose of the car, which indicates the beginning of the lane and acts as a hint to its direction. It is necessary to disambiguate which lane to detect, since several (partial) lanes may be visible on the map.

Algorithm 1 outlines the proposed approach. It receives the map points $\mathcal{M}$ and the car pose pose_{car} as an input. As the boundaries can be described naturally by two paths in a graph, we use a graph as the problem domain. After constructing the search graph, we enumerate feasible candidate solutions using path pair enumeration. In the second step, we select the most likely candidate. Our proposed algorithm is deterministic and also complete due to exhaustive enumeration.

In the following sections, we describe each step of the proposed algorithm.

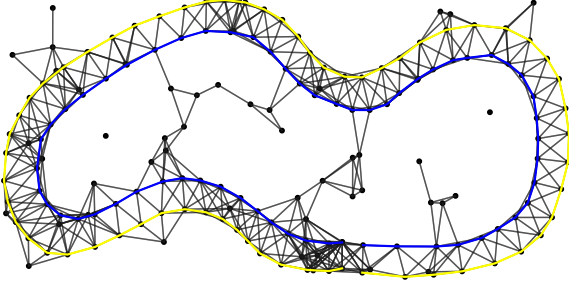
IV. GRAPH SEARCH FOR LANE DETECTION

A. Search graph construction

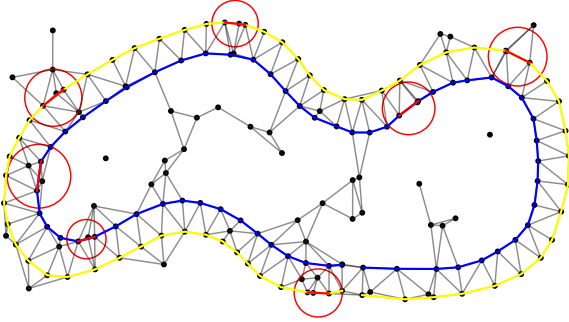
In the first step, we construct a search graph from the map with the vertices representing every map point. Edges are inserted between every pair of vertices which can be connected in a boundary. The first constraint d_{max} is used here, which states the maximum distance two points can have for a connection. Specifically for the FSD-competition, this constraint is stated in a rulebook [27] to be 5m, i.e. two consecutive points of a boundary cannot be further away than 5m. The search graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is therefore a simple and undirected graph constructed from the map $\mathcal{M}$ consisting of 2D-points $p_1, \dots, p_N \in \mathbb{R}^2$. It is defined by its vertex- and edges-sets $\mathcal{V}, \mathcal{E}$ as:

$$\mathcal{V} = \{1, \dots, N\} \quad (1)$$

$$\mathcal{E} = \{\{i, j\} \in \mathcal{V}^2 \mid \|p_i - p_j\|_2 \leq d_{max}, i \neq j\} \quad (2)$$



(a) Proposed search graph construction



(b) Search graph construction using the Delaunay-triangulation

Figure 3: We propose to construct the search graph simply by adding an edge between all points closer than d_{max} (e.g. 5m) (a). The Delaunay-triangulation (b) is an alternative way for constructing the search-graph, but in case of false-positives, the graph has missing edges (highlighted in red) that belong to the correct solution. Thus, it is not possible to find the correct solution in this search graph. (the false-positive rate shown here is 30%).

An example of a search graph constructed in this way is illustrated in Fig. 3a.

B. Delaunay-triangulation for search graph construction

The Delaunay-triangulation has been previously proposed for construction of the search graph [4, 5, 28]. It offers the advantage that it has fewer edges than our proposed approach. Using our proposed approach, the number of edges grows quadratically in the number of vertices. On the other hand, graphs constructed by triangulation have at most $3n - 6$ edges for n vertices, linear in the number of vertices [29]. However, the main disadvantage of the search graph constructed by

triangulation is that it does not necessarily contain all edges of the correct driving lane in case of false positives (Fig. 3b). Thus, it is generally not possible to find the correct lane in this search graph, as it is not contained in it. In other words, the correct solution is immediately excluded by the construction of the search graph. We will show in section VIII-G that in practice, the theoretically lower number of edges does not lead to a significantly lower runtime of our algorithm. Our approach does not require the heuristic geometric plausibility provided by the search graph, since it explicitly uses geometric constraints.

C. Determining two starting vertices

To perform the graph search for lane detection, we determine initially two left and right starting vertices. Determining these vertices is a non-critical process and can be done via simple geometric rules which are implemented in the function `find_starting_vertices` 3: Given the car pose in the map coordinate system, the two starting vertices $s_l, s_r \in \mathcal{V}$ have to be close to the car, thus we conduct a search within a maximum radius such as 2 m. Additionally, the left starting vertex must have a positive angle, the right a negative angle relative to the car heading vector. From multiple pairs that meet these criteria, we select the one that is the most symmetrical with respect to the line defined by the position and direction vector of the car.

D. Enumeration of path pairs

Given two starting vertices, we can formulate the problem of finding lane candidates as path pair enumeration. We enumerate pairs of simple paths, i.e. a sequence of vertices where no vertex occurs more than once. First, we define the solution set as follows:

Definition 1 (Solution set of path pairs). Let the set of all paths in the graph $\mathcal{G}$ starting from vertex s_l be $\mathcal{L}$ and starting from s_r be $\mathcal{R}$. The solution set $\mathcal{S}$ for driving lane detection is the set of path pairs $\mathcal{S} := \mathcal{L} \times \mathcal{R}$. The feasible solution set $\mathcal{S}_c$ is $\mathcal{S}_c := \{P \in \mathcal{S} \mid \text{CD}(P)\}$.

The solution set $\mathcal{S}$ is reduced further by imposing constraints using the predicate $\text{CD}(\cdot)$ which returns true if the lane represented by the path pair satisfies all constraints and false otherwise.

Algorithm 2 enumerates the feasible set $\mathcal{S}_c$ of path pairs by using a modified depth-first-search (DFS) that simultaneously searches both paths and incorporates heuristics as well as constraint satisfaction checking. It takes worst-case time $\mathcal{O}(|V|!)$ ($\mathcal{O}(5^{2|V|})$ with planar graphs) and $\mathcal{O}(|V| \cdot |E|)$ space. Despite the prohibitive worst-case runtime complexity, in practice the algorithm commonly is able to fully enumerate $\mathcal{S}_c$ due to the drastic reduction of the solution set imposed by the geometric constraints, most paths are not visited as the search backtracks early.

In general, we could also enumerate $\mathcal{L} \times \mathcal{R}$ by enumerating $\mathcal{L}$ and $\mathcal{R}$ separately (searching one path at a time) for which we would not need a modified DFS. However, as we mentioned earlier, our enumeration process is also a local optimization

Algorithm 2 Enumerate path pairs which satisfy constraints.

Input: $\mathcal{G}$: adjacency list of the graph; P : current path pair, initially $((s_l), (s_r))$ (containing only the starting vertices); V : pair of adjacency lists with visited vertices, initially $(\{\}, \{\})$; i : iteration counter, initially 0; it_{max} : iteration limit (default $it_{max} = 2500$)

Output: Set of path pairs which satisfy constraints

```

1: function EPP( $\mathcal{G}, P, V, i$ )
2:    $\leftarrow \emptyset$  % Set of all found path pairs
3:   loop % For all adjacent vertices
4:     if  $i \geq it_{max}$  then
5:       return
6:      $i \leftarrow i + 1$ 
7:      $c_s \leftarrow P[s].back()$  for  $s \in \{0, 1\}$  % curr. verts.
8:      $v_s \leftarrow V[s][c_s]$  for  $s \in \{0, 1\}$  % visited verts.
9:     % adjacent unvisited vertices
10:     $u_s \leftarrow (\mathcal{G}[c_s] \setminus v_s) \setminus P[s]$  for  $s \in \{0, 1\}$ 
11:    if  $u_0 = \emptyset \wedge u_1 = \emptyset$  then
12:      return
13:    % Choose next vertices for both sides
14:     $n_s \leftarrow NVD(P[s], u_s)$  for  $s \in \{0, 1\}$  V-
15:    if  $u_0 \neq \emptyset \wedge u_1 \neq \emptyset$  then
16:       $s \leftarrow LRD(P, n_0, n_1)$  % Choose next side V-B
17:    else  $s \leftarrow 1$  if  $u_0 = \emptyset$  else  $s \leftarrow 0$ 
18:     $P[s].push(n_s)$  % add next vertex to path
19:     $V[s][c_s].add(n_s)$  % Mark as visited
20:    if  $CD(P)$  then % If lane satisfies constraints 6
21:       $\leftarrow \cup \{P\}$ 
22:    if  $\neg TD(P, u_0 \neq \emptyset, u_1 \neq \emptyset)$  then VI-B
23:       $\leftarrow \cup EPP(\mathcal{G}, P, V, i)$ 
24:     $P[s].pop()$ 

```

step where we prioritize finding the local optimum. If we think of the problem as an optimization over two dimensions ($\mathcal{L}$ and $\mathcal{R}$), enumerating only one set of paths at a time corresponds to a search over only one dimension at a time. Since the optimal solution could be anywhere in this 2D space, searching over only one dimension at a time for the optimal solution clearly is not an efficient optimization method. Instead, multivariate optimization methods typically use a search direction (e.g. the gradient vector), leading to an efficient search for local minima. The same idea applies to our algorithm – by searching the two paths and thus over the two dimensions simultaneously, we can direct the search and find locally optimal solutions efficiently. The search direction is provided by a heuristic which indicates whether to continue searching further the left or the right path, it is named the *Left-right-decider* and explained further in section V-B.

Algorithm 2 begins by initializing an empty solution set (line 2) and then loops over all adjacent vertices (line 3). It obtains the last vertices of both paths (line 7) (which are labeled “current” as they are the current ones of DFS). It retrieves the already visited adjacent vertices of these current left and right vertices (line 8), as well as the unvisited ones which additionally are not already in the path (line 10). It then decides via a heuristic which unvisited adjacent vertices

to visit next (line 14). The function returns when all have been visited for both paths (line 12). After choosing the next vertices, the *side* variable s is chosen which indicates whether to continue searching further the left or right path. We use 0 for left and 1 for right as a convention. To not exclude paths, the next side is chosen by the heuristic (line 16) only if both paths have unvisited adjacent vertices (line 15). If all adjacent vertices have been visited for one path and therefore only the other path has unvisited vertices, the path with the unvisited vertices is chosen as the next one to be searched (line 17).

After adding the next vertex (line 18), the path pair is added to the solution set (line 21) if it satisfies the constraints (line 20). Then, the *backtracking-decider* TD decides whether the constraints can not be satisfied anymore if they are violated (line 22). In such a case, the algorithm backtracks. Otherwise, the function is called recursively to continue the search (line 23). Note that the iteration counter i has to be passed by reference unlike the other arguments to ensure the iterations are counted correctly.

E. Lanes for a closed course

Algorithm 2 does not enumerate closed driving lanes – in order to be able to represent the racetrack as a closed course, a *closing* is attempted as a post-processing step on each path pair found which closes both paths. A candidate can be closed if there are edges between the first and the last vertex of both the left and the right path. Also, the constraints must remain satisfied after closing. If such a closing is possible, the closed path pair is added to the solution set as an additional candidate.

F. Warmstarting

The enumeration of candidate solutions still suffers from combinatorial explosion limiting the search depth. In practice, this hinders detecting a complete racetrack which has an average length of 350m. The search depth achieved instead is in many cases below 100m (see section VIII-E for an evaluation). The solution to this problem is to reuse the most likely driving lane found in the last run of the algorithm for starting the enumeration, i.e. warmstarting. The warmstarting causes the enumeration algorithm to start with the most likely solution found (i.e. after running CLC 1 once). Here we use the fact that the map is initially not fully visible, the visible lane is commonly below these 100m which allows the algorithm to still detect the complete lane. Warmstarting causes the lane to be constructed incrementally over only a few runs of CLC for many race track layouts. Overall, warmstarting thus enables a low-latency and real-time lane detection and at the same time completes the detected lane as the map is built. This allows for global race line optimization for minimizing the lap time [30].

V. SE RCH HEURISTICS

In this section, we explain in detail how the enumeration algorithm 2 uses heuristics to guide the graph search, in particular we define the functions LRD and NVD . The two heuristics used ensure that during the limited number of iterations, locally optimal lane candidates are searched first.

. Next-vertex-decider

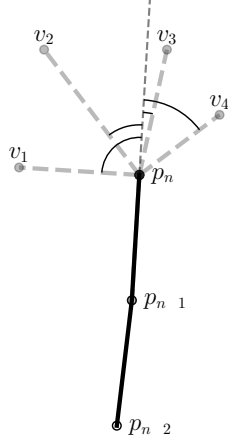


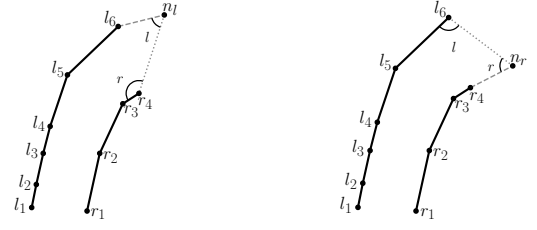
Figure 4: Next-vertex decider: For the last point p_n of either the left or right boundary we choose an adjacent vertex ($v_1, \dots, v_4$) to visit next. In this example, v_3 is chosen as the next vertex adjacent to p_n since the absolute angle between the line segments $\overline{p_{n-1}p_n}$ and $\overline{p_nv_3}$ is the smallest from all segments to possible adjacent vertices.

The next-vertex-decider selects the best adjacent vertex to visit next. It is always queried for the last vertex of both the left and right path. The idea of this heuristic is to predict straight lines based on the current direction, i.e. to extrapolate the boundary direction. For this, it considers angles of consecutive line segments of one boundary. The last vertex of a path corresponds to the last point p_n of a polygonal chain $P = (p_1, \dots, p_n)$ and has unvisited adjacent vertices $\mathcal{U}$. For every adjacent vertex $u \in \mathcal{U}$, we compute the absolute angle between the last segment $\overline{p_{n-1}p_n}$ of the boundary and the segment $\overline{p_np_u}$ from p_n to p_u , the point corresponding to the unvisited vertex u . We then select the adjacent vertex which minimizes this angle (Fig. 4). Consequently, the best next vertex for the path P is chosen from the set of vertices $\mathcal{U}$ by the next-vertex decider NVD with:

$$\text{NVD}(P, \mathcal{U}) = \arg \min_{u \in \mathcal{U}} |\angle(\overline{p_{n-1}p_n}, \overline{p_np_u})| \quad (3)$$

This is clearly a greedy heuristic which considers both boundaries independently and therefore may make suboptimal choices for the overall lane. Despite the simplicity of the approach, it proves to be effective and is also computationally cheap. Note that we use memoization to increase the execution speed as this heuristic is usually queried multiple times for the same vertex.

Initially, the polygonal chain is only a single point and therefore the segment $\overline{p_{n-1}p_n}$ is not defined. In this case we use the heading vector of the car instead of this segment. This is necessary as the search direction is ambiguous otherwise – in case of a closed racetrack, the lane can either start in the car heading direction or in the direction opposite to it.



(a) Potential next vertex for the left path. (b) Potential next vertex for the right path.

Figure 5: Left-right-decider: The left-right decider decides during the enumeration of path pairs whether a next vertex should be added to the left path (a) or to the right path (b). It is a geometric heuristic that tries to keep both paths equally *advanced* by minimizing the deviation $|l - r|$. In this example, selecting the right vertex leads to a smaller deviation, therefore the left-right decider chooses the right side and the path pair becomes (b) in the next iteration.

B. Left-right-decider

The Left-right-decider LRD is a heuristic that determines during the path pair enumeration whether it is best to continue searching the left or the right path. The heuristic is based on the geometric rule to try to maintain both polygonal chains (which correspond to the paths) equally *advanced* (Fig. 5). The previously described NVD-heuristic chooses two adjacent vertices n_0 (left) and n_1 (right), LRD chooses then which one of them to add to the path pair². It outputs the side s whether to add a vertex to the left or right path, the path on the other side remains unchanged. Therefore, if we would decide extending the left path and adding n_0 , we obtain the path pair $P'_1 = ((l_1, \dots, l_n, n_0), (r_1, \dots, r_m))$, whereas if we decide for the right path and add n_1 , we obtain the path pair $P'_2 = ((l_1, \dots, l_n), (r_1, \dots, r_m, n_1))$. LRD is now simply a greedy heuristic which chooses the side that leads to the more likely one of these two path pairs.

The heuristic works by comparing both potential path pairs P'_1 and P'_2 by computing two angles. It computes the two angles between the last line segments of both polygonal chains and the line segment connecting the last points of both polylines. The two angles l and r are defined given the left and right polygonal lines $(l_1, \dots, l_n), (r_1, \dots, r_m)$ as:

$$\begin{aligned} l &= \angle(\overline{l_{n-1}l_n}, \overline{l_n r_m}) \\ r &= \angle(\overline{r_{m-1}r_m}, \overline{r_m l_n}) \end{aligned} \quad (4)$$

The more equally advanced both polylines are, the more similar the two angles l and r are ($|r - l|$ is smaller). These two angles are computed for both lanes P'_1 and P'_2 , for P'_1 they are l_1, r_1 , for P'_2 they are l_2, r_2 . LRD is a function

²Note that we cannot simply add both vertices to the path pair at the same time, doing so would result in always enumerating path pairs of equal length which is only a subset of the solution set defined in Def. 1.

which chooses the side corresponding to the lane with smaller $|r - l|$:

$$\text{LRD}(P, n_0, n_1) = \begin{cases} 0 \text{ (left)} & | \frac{1}{r} - \frac{1}{l} | < | \frac{2}{r} - \frac{2}{l} | \\ 1 \text{ (right)} & \text{otherwise} \end{cases} \quad (5)$$

For the special case in which initially both polygonal lines have zero line segments, an arbitrary side may be chosen. Note that another special case occurs when the constraint decider may determine that the current lane does not satisfy the constraints and they can only be satisfied by extending a specific boundary (either left or right). In such a case the LR-decider simply chooses this side which can lead to constraint satisfaction. In such a case, the heuristic therefore tries to leave the unfeasible region as fast as possible.

VI. GEOMETRIC LANE CONSTRAINTS

In this section, we define the geometric constraints imposed on lanes as well as the two functions $\text{CD}(\cdot)$ and $\text{TD}(\cdot)$ (see VI-B) used by algorithm 2.

We impose the following constraints:

- 1) Two points of the same boundary have a maximum distance d_{max} ($d_{max} = 5.5m$)
- 2) C_{seg} : The absolute angle between two consecutive line segments of a boundary does not exceed a maximum ϕ_{max} ($\phi_{max} = 90^\circ$)
- 3) C_{poly} : The driving lane polygon is simple, i.e. does not intersect itself
- 4) C_{width} : The driving lane width is above w_{min} and below w_{max} ($w_{min} = 2.5m$ and $w_{max} = 6.5m$)

The predicate indicating constraint satisfaction simply requires the satisfaction of all constraints:

$$\text{CD}(P) = C_{seg}(P) \wedge C_{width}(P) \wedge C_{poly}(P) \quad (6)$$

These constraints guarantee the output of a geometrically sound lane. One major consideration of these constraints is whether, once they are violated, they can be satisfied anymore by extending the lane. One insight into the efficiency of our lane detection algorithm is that in most cases, when a constraint is violated, it cannot be satisfied anymore. This allows for pruning the search space by backtracking. Note that, specific to the FSD competition, some of these constraints are explicitly mentioned in a rulebook, for example the maximum distance between two points, as well as the lane width constraint [27, pp. 14-16]³. Recall that the first constraint is already satisfied by the graph construction (see IV-) and it therefore does not need to be checked by $\text{CD}(\cdot)$. All other geometric constraints must be checked for every new path pair P . For this, we use efficient online algorithms that only perform computations that have not been done before.

The constraints C_{seg} and C_{poly} can be ensured as follows: For C_{seg} , each time a new vertex and thus a new segment is added to the polygonal chain, we compute the absolute angle to the previous segment. If it is below ϕ_{max} or there is no previous segment, the constraint C_{seg} is satisfied. For C_{poly} ,

we define the polygon of the driving lane by the sequence of points of the left boundary concatenated with the points of the right boundary in reversed order. To check whether it intersects itself, for each new segment we check for intersection with any other segment. If no segments intersect any other, the constraint is satisfied.

In the following subsection, we will focus on the lane width constraint C_{width} . It requires defining the width of the lane bounded by two polygonal chains, which is rather non-trivial and is also further complicated by the requirement of fast online-algorithms.

. Driving lane width

In this subsection we outline the proposed approach for efficiently calculating the lane width bounded by two polygonal chains. The general idea of the lane-width calculation is to establish a set of corresponding points which lie on the left and right boundary and then define the width as the Euclidean distance between them. The main task is therefore to establish these correspondences.

The distance between two polygonal chains is a well studied topic in computational geometry with approaches such as the *Fréchet distance* [31, 32]. Another application is morphing between polygonal curves for which a *width* between the curves has been considered explicitly in [33]. All these approaches work by establishing corresponding points between both curves with reparametrization functions, named *matchings*. The Fréchet matching (leading to the Fréchet distance) however is not unique and not every solution is suitable as a calculation of the width. A solution to the problem of uniqueness was proposed in [32] where *locally correct* Fréchet matchings were considered, but the algorithm is difficult to implement and would likely be too slow for our application.

We propose therefore a new algorithm for lane width calculation which meets the following criterias:

- 1) It is a natural way for measuring the lane width
- 2) Efficient online algorithms for computation can be derived
- 3) It allows for an analysis whether the lane width constraint can be satisfied anymore once it is violated.

The third criterion states whether and when once the lane width is outside of bounds and thus the constraint is violated, it can be satisfied again by extending the lane. This is important as it allows the usage of backtracking when the constraint cannot be satisfied anymore and is thus essential for the efficiency of the graph search algorithm.

We first make some preliminary definitions. The length of a polygonal chain with N points is defined as the sum of the length of its line segments:

$$l_p = \sum_{i=1}^{N-1} \|p_{i+1} - p_i\|_2 \quad (7)$$

One way of describing a polygonal chain is as a curve $[0, l_p] \rightarrow \mathbb{R}^2$, mapping the progress along the curve to a 2D-point. We will use the slightly different parametrization for a polygonal chain where the domain depends on the number of points rather than the length of the curve: $P : [0, |P| - 1] \rightarrow$

³Note that, although the rulebook only states a minimum lane width, in practice we can also assume an upper bound.

², where $|P|$ is the number of points of the polygonal chain. To define $P(x)$, we split x in $x = i + \lambda$, the integer part $i \in \{0, 1, \dots, |P| - 1\}$ which is simply the point index. The fractional part $\lambda \in [0, 1[$ states the progress along the i -th line segment. Given the sequence of points $(p_0, \dots, p_{|P|-1})$ of the polygonal chain, we define $P(i + \lambda) = (1 - \lambda)p_i + \lambda p_{i+1}$. The set of points $P(x)$ for $x \in [i, i + 1]$ denotes the i -th segment of P .

We use this parametrization for both the left and right boundary, $L : [0, |L| - 1] \rightarrow \mathbb{R}^2$ and $R : [0, |R| - 1] \rightarrow \mathbb{R}^2$. We define the sequence of *matching points* as $M_{ps} = \{(u_i, v_i) \in [0, |L| - 1] \times [0, |R| - 1]\}_{i=0}^{|M| - 1}$, consisting of pairs of parameters for the left and right polygonal chains [32]. The sequence $\{(L(u_i), R(v_i)) \mid (u_i, v_i) \in M_{ps}(L, R)\}_{i=0}^{|M_{ps}| - 1}$ is a sequence of endpoints of line segments which we call the *matching lines*, the length of these segments is the driving lane width (see Fig. 6).

1) *Establishing a matching*: The idea for an efficient algorithm calculating these matching points is to use a nearest-neighbor search (NN-search) for closest line segments between the left and right polygonal chain. To ensure that every vertex is matched as well, we also perform nearest neighbor search with vertices as a query. We use analytic formulas for calculating the shortest line segment between two line segments.

First, we define the search space $\Omega_i(k, s)$ for arguments a and b to both polygonal chains. It depends on whether we use the i -th vertex ($k = 0$) or the i -th segment ($k = 1$) as a query and whether we use the left polygonal chain as a query with the right one as target ($s = 0$) or vice-versa ($s = 1$):

$$\Omega_i(k, s) = \begin{cases} \{i\} \times [0, |R| - 1] & \text{if } s = 0, k = 0 \\ [i, i + 1] \times [0, |R| - 1] & \text{if } s = 0, k = 1 \\ [0, |L| - 1] \times \{i\} & \text{if } s = 1, k = 0 \\ [0, |L| - 1] \times [i, i + 1] & \text{if } s = 1, k = 1 \end{cases} \quad (8)$$

This search space can be interpreted as follows: The nearest-neighbor search always has as a target domain the complete polygonal chain, i.e. the intervals $[0, |L| - 1]$ and $[0, |R| - 1]$. The query for the NN-search can either be a single line segment of the polygonal chain ($[i, i + 1]$), or a single point, i.e. $\{i\}$, for i being an integer.

We define then the nearest-neighbor search as yielding the matching points $M(L, R, s, k)$ between L and R :

$$M(L, R, k, s) = \left\{ (u_i, v_i) = \arg \min_{(a, b) \in \Omega_i(k, s)} \|L(a) - R(b)\|_2 \right\}_{i=0}^{N_m(k, s) - 1} \quad (9)$$

$N_m(k, s)$ is the number of matching points, it is defined as:

$$N_m(k, s) = \begin{cases} |L| & k \text{ if } s = 0 \\ |R| & k \text{ if } s = 1 \end{cases} \quad (10)$$

When using segments as a query ($k = 1$) instead of vertices ($k = 0$), there is one matching point less due to a polygonal

chain having one segment less than it has points, this is why we subtract k . Also, depending on whether the polyline L or R is the query, the number of matching points depends either on $|L|$ or on $|R|$.

The proposed method for computing the matching points M_{ps} between the two boundaries L and R is by combining nearest points-to-segments and segments-to-segments and using L and R as query as well as a target:

$$M_{ps}(L, R) = \bigcup_{s \in \{0, 1\}} M(L, R, s, k) \quad (11)$$

The lane width W between the two boundaries L, R is defined simply as the Euclidean distance between corresponding points:

$$W(L, R) = \{\|L(u_i) - R(v_i)\|_2, (u_i, v_i) \in M_{ps}(L, R)\} \quad (12)$$

The lane width constraint predicate $C_{width}(P)$ requires that the lane width is everywhere above w_{max} and below w_{min} :

$$C_{width}(P) = \bigwedge_{w \in W(L, R)} w_{min} < w_i < w_{max} \quad (13)$$

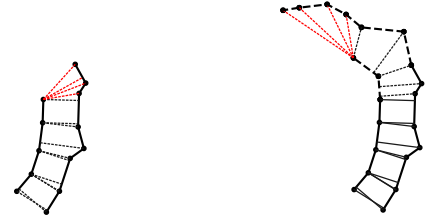


Figure 6: Left: To obtain the driving lane width, *matching lines* (thin dashed lines) are computed between the two lane boundaries (solid lines). The lengths of these matching lines define the driving lane width. The online algorithm may recompute all the *mutable* matching lines (red) which may overestimate the lane width. Right: After extending the boundaries, new matching lines are computed (dashed), replacing the mutable ones. The set of *fixed* matching lines (black dashed and solid) is only extended, the fixed matching lines from the previous iteration (black solid) are never recomputed.

2) *Online-algorithm for lane width calculation*: Instead of applying formula 11 every time for every new pair of boundaries, we propose the following online-algorithm (Fig. 3). The idea is to divide the set of matching points into two sets, the set of *fixed* matching points which are never recomputed but instead only new ones may be added. The second set of *mutable* matching points may be recomputed each time the boundaries are extended (Fig. 6). This split into two sets not only allows for greater efficiency but also for backtracking. If the lane width does not satisfy constraints and the violating matching point belongs to the set of fixed matching points, we can backtrack since this matching will not be recomputed. See lemma 3 for a detailed analysis of the backtracking criteria.

We now describe in detail Algorithm 3. The computation of matching points starts from the last matching point from the

Algorithm 3 Online algorithm for lane width calculation

Input: L, R : two lane boundaries (polygonal chains), M_{fixed}^{t-1} : fixed matching points from last iteration, sorted
Output: M_{fixed}^t, M_{mut}^t : Updated fixed and mutable matching points

```

1: function ONLINELW( $L, R, M_{fixed}^{t-1}$ )
2:   if  $M_{fixed}^{t-1} = \emptyset$  then
3:      $u_s, v_s \leftarrow (0, 0)$ 
4:   else
5:      $u_s, v_s \leftarrow M_{fixed}^{t-1}.\text{back}()$ 
6:    $M' \leftarrow$  compute matching for  $L, R$  starting from  $u_s, v_s$  (Eq. 11)
7:   sort  $M'$  and split into  $M'_{fixed}$  and  $M'_{mut}$ 
8:    $M_{fixed}^t \leftarrow M_{fixed}^{t-1} \cup M'_{fixed}$ 
9:   return  $M_{fixed}^t, M_{mut}^t$ 

```

previous iteration (line 5), or the beginning of the boundaries in case it is the first iteration (line 3). After performing the nearest-neighbor search for matching points (line 6), the set of computed matching points is split into the fixed and mutable ones based on the first matching point which matches at least one end of either boundary (i.e. it is either $(|L| - 1, v)$ or $(u, |R| - 1)$). All matching points which come before this matching point are fixed, all which come after it are mutable. For this, an order of matching points needs to be established. Since the search in 11 does not guarantee the matching points to be monotone, i.e. $(u_i \leq w_i \wedge v_i \leq x_i) \vee (u_i \geq w_i \wedge v_i \geq x_i) \forall ((u_i, v_i), (w_i, x_i)) \in M_{ps}^2$ generally does not hold, we establish a lexicographic order.

After splitting the computed matching points and sorting, the sequence of new fixed matching points M'_{fixed} (which may be empty) is appended to the ones from the previous iteration M_{fixed}^{t-1} (line 8), which maintains the order. The constraint Eq. 13 is evaluated over both sets of matching points, i.e. M_{fixed}^t and M_{mut}^t .

B. Backtracking-decider

Note that, most of the time, the geometric constraints can no longer be satisfied once they have been violated, i.e. the graph search can immediately backtrack. However, a backtracking-decider $\text{TD}(\cdot)$ is still required because there are some exceptions to this rule. It returns true if we can backtrack and false if not. It is defined as follows. $\text{TD}(\cdot)$ generally returns false if no constraints are violated, i.e. $\neg \text{CD}(P) \implies \neg \text{TD}(P, v_l, v_r) \forall P \in \mathcal{S}, v_l, v_r \in \{\text{true}, \text{false}\}^2$. In all other cases it returns true except in the ones where the constraints C_{poly} and C_{width} still can be satisfied (see lemmata 2 and 3), in which it returns false.

VII. RANKING OF CANDIDATE DRIVING LINES

In this section we outline the second essential step of our algorithm CLC 1 after candidate solutions have been enumerated, which is choosing the most likely candidate via ranking. This task is challenging as the only measurements indicating the likelihood of a lane are the boundary points and

there can be ambiguities. One simple but effective heuristic which works well in case of no false-positives is to simply select the lane with the greatest length. Other criteria are the curvatures of the two boundaries (which should not exceed a maximum) and the similarity between the two boundaries (which should be high). These geometric criteria are however subtle and tuning handcrafted heuristics exploiting them can be difficult.

We propose therefore for this pattern recognition problem the usage of a supervised machine learning approach. Instead of assigning numerical likelihood values to each candidate lane, we use the formulation as a ranking-problem where a binary classifier chooses one candidate over another (pair-wise ranking) as more likely. This is assumed to improve generalizability over directly predicting a likelihood in case the lane length varies a lot.

The input to the neural network is an 8-dimensional feature vector derived from the driving lane. We use as features (1) the lane length (computed as the mean of the length of the two boundaries) and the number of points of the left (2) and right (3) boundary. As an estimation of the geometric implausibility, we compute the variance of the lane width (4). Also, we compute the variance of line segment lengths and the angles between consecutive line segments for both boundaries (5, 6 and 7, 8 respectively). We also tried adding the respective mean values, but no improvement was observed. These are clearly hand-crafted features, but parameter-free and thus removing overall the need for hand-tuning parameters.

The model used is a 2-layer fully-connected neural network inspired by *RankNet* [34] which is a lightweight architecture suitable for real-time inference. It has in the first layer $8 \cdot 100$ parameters and in the second layer $100 \cdot 1$ parameters, using ReLU as activation function. Similar to the RankNet-approach, the predicted output probability p_{pred} is computed from the two feature vectors x_1 and x_2 with $p_{pred} = \text{sigmoid}(\text{Net}(x_1) - \text{Net}(x_2))$ where $\text{Net}(\cdot)$ is the two-layer neural network. Binary-cross-entropy-loss is used as a loss function, for which the ground-truth probability p_{GT} is derived from the intersection over union (IoU)-values between a candidate lane polygon and the ground-truth lane polygon. For two candidates, this yields the two values $\text{IoU}_1, \text{IoU}_2$, p_{GT} is defined then as:

$$p_{GT} = \text{sigmoid}(\lambda_{\text{iou}}(\text{IoU}_1 - \text{IoU}_2)) \quad (14)$$

The factor λ_{iou} is required to increase the IoU-difference as otherwise training happens very slowly. It is a non-critical parameter, in our experiments values from $\lambda_{\text{iou}} = 10$ to $\lambda_{\text{iou}} = 150$ worked equally well. The dataset used for training consists overall of lists of enumerated candidates each time the lane detection runs. The lane detection runs continuously as the map of the environment is build and therefore has to work reliably over all partial (potentially noisy) maps. To include only significant map changes, we use as a criterion the traveled distance of the vehicle which has to be above 1m. To construct the training dataset for the ranking problem, we enumerate candidates for every such partial map which yields candidate lists. We then pair candidates and compute the ground-truth probability with Eq. 14 to obtain the dataset for the binary classification problem.

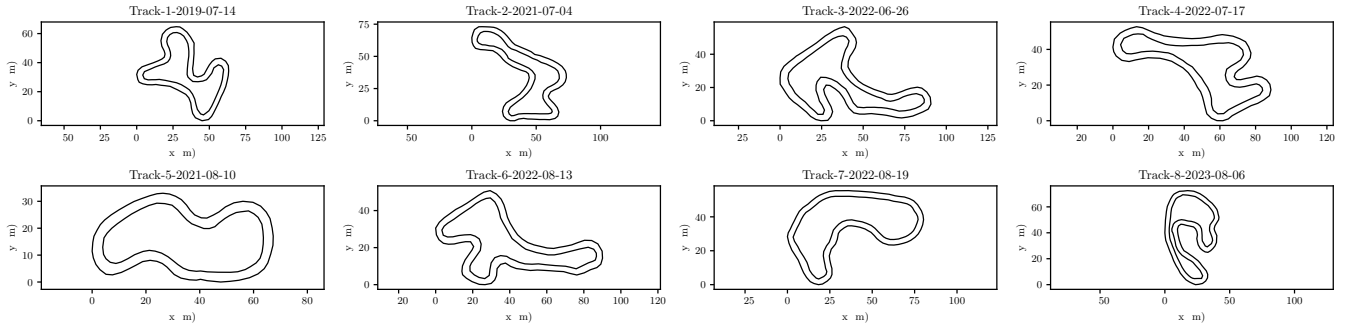


Figure 7: The dataset used for evaluation contains realistic race track layouts including curvy ones which are challenging for the lane detection.

VIII. EVALUATION

This section evaluates the proposed lane detection algorithm regarding its accuracy, efficiency and robustness against false positives. It shows the ability to detect the entire visible lane in most instances in real time. We use a dataset consisting of several race tracks built from real data with different realistic track layouts. The proposed algorithm is implemented in C++, geometric computations were performed in single-precision floating point arithmetic (instead of arbitrary-precision arithmetic) except the Delaunay-triangulation which is computed by the CGAL-library [35]. The neural network was implemented with PyTorch for training, whereas TensorFlow Lite was used for inference and evaluation, performing the inference on CPU. All experiments were executed on a PC with an Intel i7-12700K CPU with 32 GiB of RAM, comparable to the PC installed in our formula student race car.

A. Dataset

Since no open datasets are available which contain realistic race track layouts with boundaries marked by points, we created our own dataset for evaluation built from real recorded data on various race track layouts (Fig. 7). It contains for every track layout a map with the 2D-points and the trajectory driven by the car. The points were mapped by the LiDAR-based SLAM algorithm *Rao-blackwellized particle filter-SLAM* (RBPF-SLAM) [36], a variant of FastSLAM [37] that is implemented in the Mobile Robot Programming Toolkit (MRPT) library [38]. For every map, the ground-truth driving lane boundaries were annotated manually. Each time the SLAM updated the map, it also updated the car pose. These car poses are used as a starting parameter for the algorithm and are part of the dataset as well. Overall, the dataset consists of 9 tracks and a total of 1964 car poses.

B. Experiments and Metrics

We evaluate the lane detection algorithm in the same way as it is used on the racecar, every time the map is updated. To avoid the tedious task of manually annotating partial maps, we instead annotate the complete map once and then simulate partial maps using this complete map. For this, we use the known perceptual field and the car pose, and filter out the

points outside the perceptual field. We then simulate false-positives by randomly sampling from an uniform distribution in the field of perception. The LiDAR-sensor of the car has a field of perception of a half-circle in front of the car with a radius corresponding to the perception range. The noise-free maps were augmented with false positive rates of 10%, 30%, and 50%, and two different ranges of perception were simulated, 30m and 50m, resulting in a total of 6 variations of the dataset.

The neural network was trained on enumerated candidates which yielded approx. 270.000 candidates, grouped into 1964 lists. The training schedule was using 200 epochs with a batch-size of 8192 and Adam as optimizer with an learning-rate of 0.008. Since the overall architecture has only 1001 parameters, it takes only around two minutes to train on an Nvidia RTX 3070 Ti GPU.

As metrics, we use the Intersection-over-Union (IoU) between the ground-truth lane and the predicted lane. Also, we determine for each predicted lane whether it diverges from the ground-truth lane, and if so, the distance after which it does, we name it the *divergence distance*. We also compare the length of the predicted lane to the length of the ground-truth lane to identify cases where a shorter lane is predicted. If every vertex of both boundaries is predicted correctly, such a lane prediction is said to be ground-truth, i.e. exactly matching the correct lane. Additionally to stating the values of these metrics directly, we introduce categories for predictions based on these metrics for easier interpretability of the results (Fig. 8). A non-ground-truth prediction can either be too short (Fig. 8b), diverging (Fig. 8a) (which may be a critical failure case if it happens below 20m) or near-ground truth (Fig. 8c).

C. Used parameters

The maximum number of iterations it_{max} was set in all experiments to 2500 except for Fig. 11, where it was set to 20000. The values for the geometric constraints were set to $w_{min} = 2.5$ and $w_{max} = 6.5$, the maximum distance between two points to $d_{max} = 5.5$. The maximum angle between two consecutive segments was set to $\phi_{max} = 90^\circ$. These values were set such that no ground-truth lane occurring in the dataset was detected as violating the constraints, this was verified experimentally.

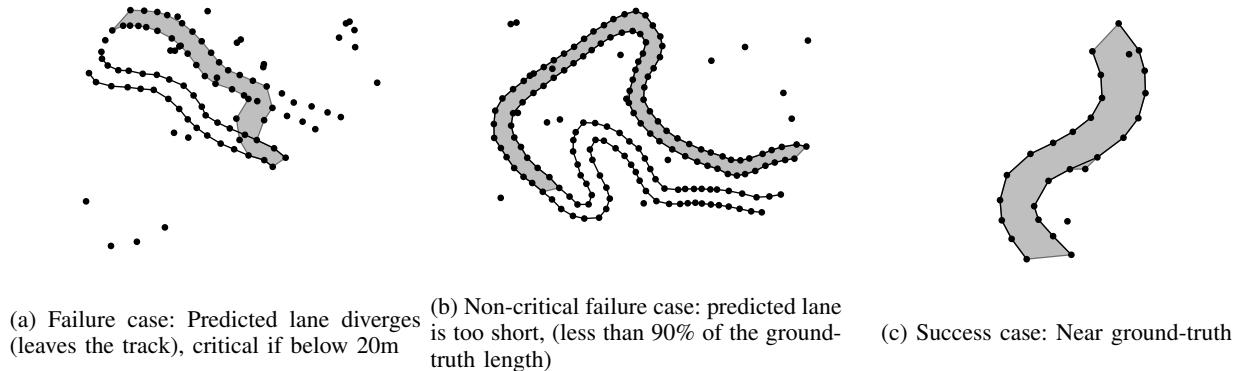


Figure 8: We classify each prediction into the following categories: Besides ground-truth predictions, the predicted lane may diverge 8a. It is critical in case it happens below 20m, as it can cause the car to leave the track. The lane can be too short 8b, this may only make the racing-line suboptimal. If the predicted lane is neither of these, it is a *near ground-truth* solution 8c. Such predictions typically have a high overlap with the ground-truth lane. (The gray polygon is the prediction, the black boundaries are ground-truth.)

FP-Rate	Success-Cases (%)					Failure-cases (%)		Sum: Success (%)	Failure (%)	mean-IoU (%)
	GT	Near-GT	Too short	Diverging	2 m	Diverging	2 m			
0 %	34.5	61.9	0.7	2.2		0.6		99.4	0.6	96.9
10 %	24.9	70.9	0.5	2.9		0.8		99.2	0.8	96.0
30 %	9.3	85.1	0.8	4.1		0.7		99.3	0.7	93.2
50 %	2.1	70.9	2.6	19.3		5.0		95.0	5.0	79.5

Table I: analysis of the lane prediction accuracy: Rates of different prediction categories depending on the false-positive rate. With zero false-positives, the ground-truth predictions are dominant, but they decrease rapidly as the number of false-positives increase. Critical failures where the driving lane diverges below 20m only occur in about 1% of instances, except at extreme FP-rates of 50%. Predictions that are too short rarely occur overall.

D. Results

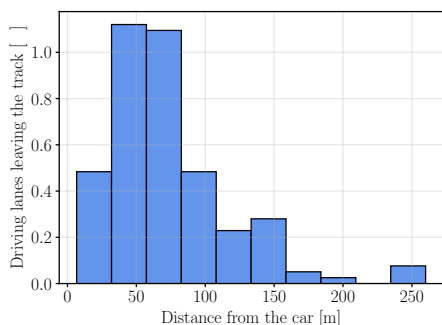


Figure 9: Distance till the divergence point in case of divergent lane predictions. The overall rate of divergent predictions is low. (Shown for FP-rates of 0% to 30%)

Table I shows the rates of various prediction categories at different false-positive rates. The rate of predicting a lane diverging below 20m is overall very low and below 1% for FP-rates of 0% - 30%. Our rate of such critical failures is 0.6%, which is lower than the method from [8] where 5% are achieved. At 50% FP-rate, which normally occurs only with bad performing perception, the critical failures rise to 2.6%, which is similar to the approach presented in [8]. See also Fig. 9 for a histogram of the divergence distances. Compared to

the methods presented in [8] and [3], our algorithm generally predicts much longer driving lanes (Fig. 12b) of up to 100m. In [3], the algorithm is limited to 15m, whereas in [8] the prediction horizon is not reported but from the limitation to 16 input points and the constrained distance between two points (which is 5m), it can be concluded that the maximum lane length can be 40m (in case of no false-positives and a straight lane). Qualitative results are shown in Fig. 10.

E. Enumeration Evaluation

FP-Rate	Instances with complete enumeration (%)
0 %	46.4
10 %	34.4
30 %	13.4
50 %	2.3

Table II: Candidate enumeration completeness: Especially at a low false-positive rate, the solution set is completely enumerated in many instances, allowing finding the global optimum of the likelihood function.

If an incorrect driving lane is predicted, we distinguish between two possible causes: either the correct driving lane was not found during enumeration, or the ranking selected the wrong candidate as the most likely one (the second step). In the following, we focus on the first failure mode

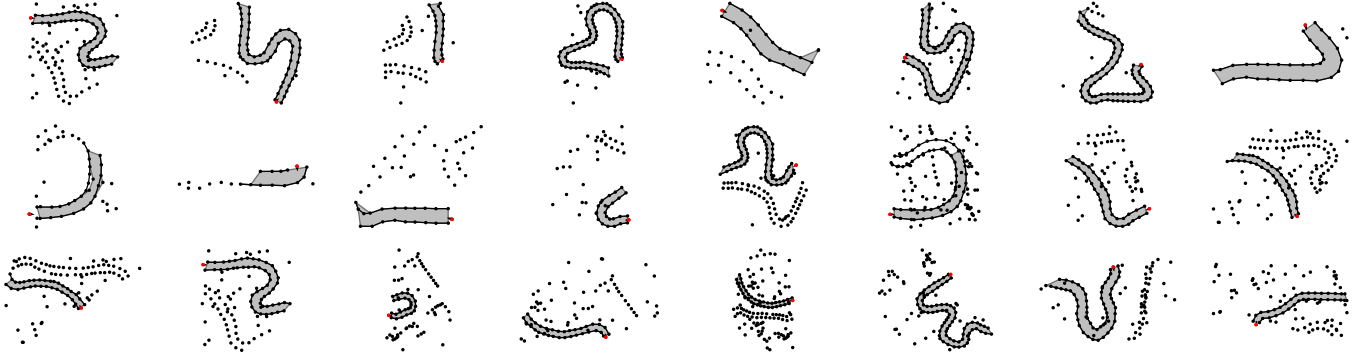


Figure 10: Qualitative results of successful lane detection: Our algorithm is robust to high amounts of false-positives, disconnected parts of the lane and is able to detect curvy, long-horizon lanes.

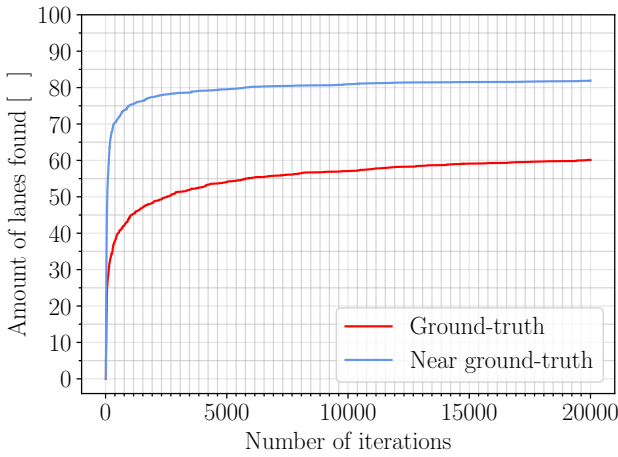
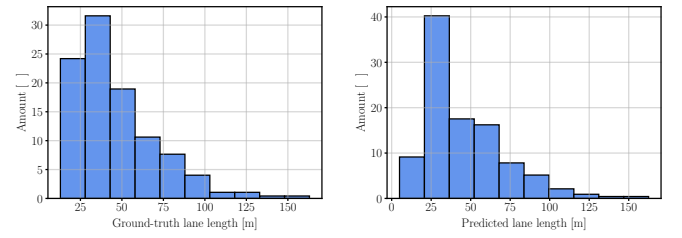


Figure 11: Iterations needed to enumerate (near) ground truth lane candidates: Most are discovered during the initial 500 iterations, demonstrating effective search heuristics. Increasing the maximum number of iterations only yields diminishing returns in terms of the amount of correct candidates found, therefore setting it to 2500 is sufficient while ensuring real-time operation. Note that although the enumeration is complete, a plateau is visible as the search becomes stuck in local minima – lanes remain which are not found even after 20000 iterations and would require a vastly higher iteration count. (Shown for FP-rates of 0% to 30%)

and evaluate the frequency of enumeration failures. First, we observe that especially with no false-positives, the enumeration is in many instances complete (Tab. II) – this allows finding the global optimum of the likelihood function, without relying on heuristics. In cases of incomplete enumeration, we rely on heuristics to find a near ground-truth candidate with as few iterations as possible. We define a near ground-truth candidate as one that has at least 98% IoU with the ground-truth driving lane. Figure 11 shows that in most instances, a ground-truth or a near-ground-truth lane is enumerated within 2500 iterations, demonstrating effectiveness of the search heuristics. At the same time, this analysis suggests a suitable value for the parameter it_{max} , the maximum number of iterations. Failure



(a) Histogram of ground-truth lane lengths. (b) Histogram of predicted lane lengths.

Figure 12: (a): Histogram of the ground-truth driving lane lengths occurring in the dataset. The histogram of the length of non-divergent predictions (b) coincides – very long driving lanes of 100m and even 150m are predicted similarly well. (Shown for FP-rates of 0% to 30%)

to enumerate a lane with at least 98% IoU occurs in 5% of instances with FP-rates of 0% with 30%.

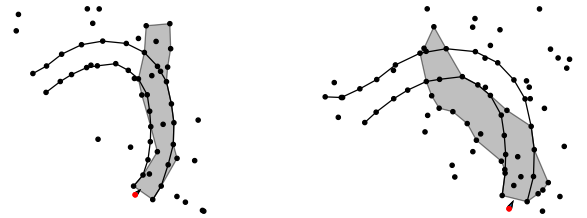


Figure 13: Typical enumeration failure cases caused by greedy heuristics: The heuristic NVD (see V-) predicts straights (left) and does not favor parallel line segments (right).

Examples of enumeration failures are shown in Fig. 13. These failures are generally caused by greedy heuristics, leading to the search becoming stuck in a local minimum.

Also, they are amplified by a high amount of false-positives. Specifically, failure mode (a) occurs as the heuristic V- always predicts straight lanes. Such a failure mode may be reduced therefore by using a heuristic which predicts the curvature. Failure (b) occurs when the heuristic V- selects the two next vertices independently, without considering the

angle between the segments and thus without favoring parallel boundaries.

F. Lane ranking evaluation

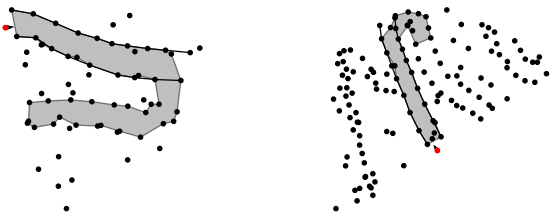


Figure 14: Typical failure cases caused by a failure of the ranking to choose the most likely lane.

We evaluated the ranking neural network on one-third of the dataset, while the remaining two-thirds were used for training, with false-positive rates of 0% to 30%. The mean Intersection over Union (mean-IoU) was used as the evaluation metric. The neural network achieved a mean-IoU of 95.10% when evaluated on the complete dataset, including the training dataset. When evaluated on the hold-out evaluation dataset, the neural network achieved only a slightly lower mean-IoU of 94.79% which demonstrates the ability to generalize well. Note that the maximum achievable IoU is not 100% due to the enumeration not being able to always find the ground-truth candidate. Instead the maximum achievable IoU were 97.99% and 97.75% for the complete and the validation dataset respectively.

We analyzed failure cases of the ranking ($\text{IoU} < 90\%$), they are mainly diverging lanes, examples are shown in Fig. 14. Most failure cases arise due to false-positives introducing ambiguity.

G. Execution time

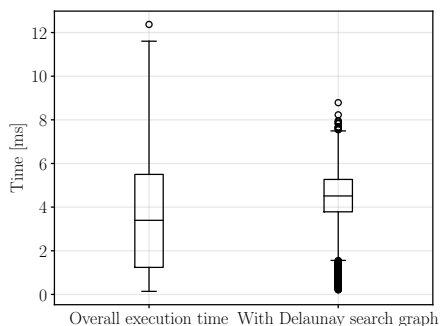


Figure 15: Overall execution time of the proposed algorithm. Using the Delaunay-triangulation for constructing the search graph does not reduce the runtime significantly although the graph is theoretically sparser. (Sample size for each box-plot is 1964)

The overall execution time of CLC is shown in Fig 15. With a median runtime of under 5ms and a maximum runtime

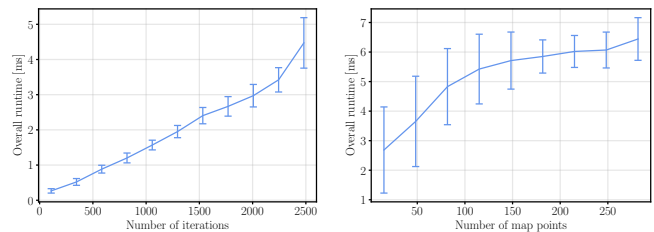


Figure 16: Runtime of CLC over the number of iterations and number of map points respectively. The runtime of CLC increases linearly with the number of iterations whereas with the number of map points the runtime increases underproportional. Overall, the number of map points has a low correlation with the runtime. (The error bars show the interval of one standard deviation, sample size is 1964).

of 12ms, our algorithm CLC is able to meet the low-latency criterion required in autonomous racing. It is also able to run after each map update which happens at the LiD R-rate of 20 Hz. When evaluating the runtime over the number of iterations performed and the number of input map points, we observe an approximately linear increase of the runtime when the iterations increase, whereas the number of map points only affects the runtime slightly (Fig. 16). Compared to the approach presented in [8] where runtimes of 71ms - 176 ms are reported, our algorithm is faster while being able to handle an input containing much more than 16 points in the input map since the runtime does not grow over-proportionally with the number of map points. We also evaluated the algorithm using search graphs constructed by Delaunay-triangulation and otherwise the same setup, where we see no significantly lower runtime. This indicates that the search is not significantly more efficient despite the higher graph sparsity. By adjusting the maximum number of iterations, we can effectively limit the execution time of CLC and guarantee its low latency.

Overall, the graph search is very efficient since it explores solutions only once and uses effective heuristics. The geometric constraints are also not too prohibitively expensive to compute when implemented with online-algorithms and short-circuiting is used.

IX. CONCLUSION AND FURTHER WORK

We presented a solution to the lane detection problem where the boundaries are sparsely marked only by 2D-points instead of continuous lines. By modeling both boundaries as paths in a graph and using exhaustive enumeration, we were able to hypothesize all possible driving lanes and thus be robust to high levels of false positives. By incorporating carefully designed geometric constraints on the candidate lanes, we were able to make the graph search efficient by pruning most of the solution set via backtracking. We presented simple and intuitive heuristics that quickly guide the graph search to locally optimal solutions, allowing a near-ground-truth solution to be found in over 50% of instances using less than 500 iterations of DFS.

Our comprehensive evaluation on many different racetrack layouts, both easy and difficult with tight hairpin turns, shows

superiority in both (1) the prediction horizon, where our approach is able to detect lanes over 100m in length and (2) in the robustness to ambiguous track layouts arising from partial map observation and false positives. Furthermore, our machine learning based approach for ranking candidate lanes effectively solves the pattern recognition problem and eliminates the need for manual tuning of heuristics. Overall, the proposed lane detection algorithm allows our racecar to drive at speeds up to 20m/s on previously unknown race tracks due to the long horizon prediction. The presented algorithm was used in the car of our Formula Student Team StarkStrom ugsburg, which participates in the most prestigious Formula Student Germany competition, driving the fastest lap times from 2021 to 2023 and even winning the competition in 2022.

Further work may include improving the graph search by (1) learning search heuristics, e.g with graph neural networks such as in [25], and (2) using exploration-vs-exploitation heuristics to help escape sub-optimal local minima. Another improvement may be to use boundary points directly as an input to the neural network instead of using hand-crafted features.

KNOWLEDGEMENTS

We would like to thank the whole formula student team StarkStrom ugsburg for their support in maintaining and improving the race car and helping gather real data during test drives which made the evaluation possible.

REFERENCES

- [1] J. Betz, H. Zheng, . Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, “Autonomous vehicles on the edge: Survey on autonomous vehicle racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [2] Tanzmeister, Georg and Friedl, Martin and Lawitzky, Andreas and Wollherr, Dirk and Buss, Martin, “Road course estimation in unknown, structured environments,” in *2013 IEEE Intelligent Vehicles Symposium (IV)*, 2013, pp. 630–635.
- [3] L. Andresen, . Brandemuehl, . Honger, B. Kuan, N. Vödisch, H. Blum, V. Reijgwart, L. Bernreiter, L. Schaupp, J. J. Chung, M. Burki, M. R. Oswald, R. Siegwart, and . Gawel, “Accurate Mapping and Planning for Autonomous Racing,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 4743–4749.
- [4] J. Kabzan, M. de la Iglesia Valls, V. Reijgwart, H. F. C. Hendriks, C. Ehmke, M. Prajapat, . Bühler, N. B. Gosala, M. Gupta, R. Sivanesan, . Dhall, E. Chisari, N. Karnchanachari, S. Brits, M. Dangel, I. Sa, R. Dubé, . Gawel, M. Pfeiffer, . Liniger, J. Lygeros, and R. Y. Siegwart, “MZ Driverless: The full autonomous racing system,” *Journal of Field Robotics*, vol. 37, pp. 1267 – 1294, 2019.
- [5] . Alvarez, N. Denner, Z. Feng, D. Fischer, Y. Gao, L. Harsch, S. Herz, N. L. Large, B. Nguyen, C. Rosero, S. Schaefer, . Terletskiy, L. Wahl, S. Wang, J. Yakupova, and H. Yu, “The Software Stack That Won the Formula Student Driverless Competition,” 2022. [Online]. available: <https://arxiv.org/abs/2210.10933>
- [6] ssidiq, bdulhakam. M. and Khalifa, Othman O. and Islam, Md. Rafiqul and Khan, Sheraz, “Real time lane detection for autonomous vehicles,” in *2008 International Conference on Computer and Communication Engineering*, 2008, pp. 82–88.
- [7] Huang, Jing and Choudhury, Pallab K. and Yin, Song and Zhu, Lingyun, “Real-time road curb and lane detection for autonomous driving using lidar point clouds,” *IEEE Access*, vol. 9, pp. 144 940–144 951, 2021.
- [8] K. Brandes, . Wang, and R. Shah, “Robust lane detection with binary integer optimization,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 229–235.
- [9] S. M. LaValle, J. J. Kuffner, B. Donald *et al.*, “Rapidly-exploring random trees: Progress and prospects,” *Algorithmic and computational robotics: new directions*, vol. 5, pp. 293–308, 2001.
- [10] Vonásek, Vojtěch and Faigl, Jan and Krajník, Tomáš and Přeučil, Libor, “RRT-path – a guided rapidly exploring random tree,” in *Robot motion and control 2009*. Springer, 2009, pp. 307–316.
- [11] L. Noakes and T. Popiel, “Geometry for robot path planning,” *Robotica*, vol. 25, pp. 691 – 701, 2007.
- [12] Yue Wang and Dinggang Shen and Eam Khwang Teoh, “Lane detection using spline model,” *Pattern Recognition Letters*, vol. 21, no. 8, pp. 677–689, 2000.
- [13] Yu, B. and Jain, .K., “Lane boundary detection using a multiresolution hough transform,” in *Proceedings of International Conference on Image Processing*, vol. 2, 1997, pp. 748–751 vol.2.
- [14] . Kaske, D. Wolf, and R. Husson, “Lane boundary detection using statistical criteria,” in *International Conference on Quality by Artificial Vision, QC V9*. Citeseer, 1997, pp. 28–30.
- [15] J. Beck and C. Stiller, “Non-parametric lane estimation in urban environments,” in *2014 IEEE Intelligent Vehicles Symposium Proceedings*, 2014, pp. 43–48.
- [16] Zhang, Jie and Xu, Yi and Ni, Bingbing and Duan, Zhenyu, “Geometric constrained joint lane segmentation and lane boundary detection,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.
- [17] Qin, Zequn and Wang, Huanyu and Li, Xi, “Ultra Fast Structure-aware Deep Lane Detection,” in *The European Conference on Computer Vision (ECCV)*, 2020.
- [18] Tabelini, Lucas and Berriel, Rodrigo and Paixao, Thiago M. and Badue, Claudine and De Souza, Iberto F. and Oliveira-Santos, Thiago, “Keep your eyes on the lane: Real-time attention-guided lane detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 294–302.
- [19] C. Li, J. Shi, Y. Wang, and G. Cheng, “Reconstruct from top view: 3d lane detection approach based on geometry structure prior,” in *2022 IEEE/CVF Conference*

- on Computer Vision and Pattern Recognition Workshops (CVPRW), 2022, pp. 4369–4378.
- [20] S. Sultana, B. Ahmed, M. Paul, M. R. Islam, and S. Ahmad, “Vision-based robust lane detection and tracking in challenging conditions,” *IEEE Access*, vol. 11, pp. 67 938–67 955, 2023.
- [21] Q. Li, L. Chen, M. Li, S.-L. Shaw, and J. Nüchter, “Sensor-fusion drivable-region and lane-detection system for autonomous vehicle navigation in challenging road scenarios,” *IEEE Transactions on Vehicular Technology*, vol. 63, no. 2, pp. 540–555, 2014.
- [22] J. Gasparetto, P. Boscariol, J. Lanzutti, and R. Vidoni, *Path Planning and Trajectory Planning Algorithms: General Overview*. Cham: Springer International Publishing, 2015, pp. 3–27.
- [23] D. Dolgov, D. Thrun, S. Sebastian, and M. Montemerlo, M. Diebel, J. Michael, “Path Planning for Autonomous Driving in Unknown Environments,” in *Experimental Robotics*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 55–64.
- [24] A. Heilmeier, A. Wischnewski, L. Leonhard, H. Hermansdorfer, J. Betz, M. Lienkamp and B. Lohmann, “Minimum curvature trajectory planning and control for an autonomous race car,” *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020. [Online]. available: <https://doi.org/10.1080/00423114.2019.1631455>
- [25] K. Osanlou, J. Bursuc, C. Guettier, T. Cazenave, and E. Jacopin, “Optimal solving of constrained path-planning problems with graph convolutional networks and optimized tree search,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 3519–3525.
- [26] Z. Li, Q. Chen, and V. Koltun, “Combinatorial optimization with graph convolutional networks and guided tree search,” in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018. [Online]. available: https://proceedings.neurips.cc/paper_files/paper/2018/file/8d3bba7425e7c98c50f52ca1b52d3735-Paper.pdf
- [27] “FSG 2023 Competition Handbook v1.0,” 2023. [Online]. available: https://www.formulastudent.de/fileadmin/user_upload/all/2023/important_docs/FSG23_Competition_Handbook_v1.0.pdf
- [28] P. Lu, S. Xu, and H. Peng, “Graph-embedded lane detection,” *IEEE Transactions on Image Processing*, vol. 30, pp. 2977–2988, 2021.
- [29] S.-W. Cheng, T. K. Dey, and J. R. Shewchuk, *Delaunay Mesh Generation*. Chapman and Hall/CRC, 2013. [Online]. available: <http://www.cs.berkeley.edu/~jrs/papers/meshbook/chapter2.pdf>
- [30] Christ, F. and Wischnewski, J. and Heilmeier, J. and Lohmann, B. , “Time-optimal trajectory planning for a race car considering variable tyre-road friction coefficients,” *Vehicle System Dynamics*, no. Vol. 59, Issue 4, pp. pp. 588–612, Apr 2021.
- [31] H. It and M. Godau, “Computing the Fréchet distance between two polygonal curves,” *Int. J. Comput. Geom. appl.*, vol. 5, pp. 75–91, 1995.
- [32] K. Buchin, M. Buchin, W. Meulemans, and B. Speckmann, “Locally correct fréchet matchings,” in *Algorithms – ES 2012*, L. Epstein and P. Ferragina, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 229–240.
- [33] J. Efrat, S. Har-Peled, L. Guibas, and T. Murali, “Morphing between polylines,” in *Proceedings of the 12th Annual CM-SI M Symposium on Discrete Algorithms*, ser. Proceedings of the Annual CM-SI M Symposium on Discrete Algorithms, 2001, pp. 680–689.
- [34] C. J. C. Burges, T. Shaked, E. Renshaw, J. Lazier, M. Deeds, N. Hamilton, and G. N. Hullender, “Learning to rank using gradient descent,” *Proceedings of the 22nd international conference on Machine learning*, 2005.
- [35] S. Hert and M. Seel, “dD convex hulls and Delaunay triangulations,” in *CG L User and Reference Manual*, 4.3 ed. CG L Editorial Board, 2013. [Online]. available: <http://doc.cgal.org/4.3/Manual/packages.html#PkgConvexHullDSummary>
- [36] J.-L. Blanco, J. Gonzalez, and J.-J. Fernandez-Madrigal, “An optimal filtering algorithm for non-parametric observation models in robot localization,” in *2008 IEEE International Conference on Robotics and Automation*, 2008, pp. 461–466.
- [37] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, “FastSL M: Factored Solution to the Simultaneous Localization and Mapping Problem,” in *II/I I*. MIT Press / The MIT Press, 2002, pp. 593–598.
- [38] MRPT-library. accessed: 2023-11-12. [Online]. available: <https://docs.mrpt.org/reference/latest/>

Ivo Ivanov Ivo Ivanov received his B.Sc degree in computer science from the Technical University of Applied Sciences Augsburg (Germany) in 2022 and is currently a master’s student in computer science. He was an active member of the Formula Student team StarkStrom Augsburg from 2019 to 2021. Currently he focuses on research in perception and machine learning for autonomous driving.

Carsten Markgraf Prof. Dr.-Ing. Carsten Markgraf finished his studies of Electrical Engineering in 1997 at the University of Hannover (Germany). As a research assistant he was cooperating with VW to analyze infrastructure-based technologies for the automation of vehicle test tracks. After he finished his PhD on the topic “Automated Driving with Magnetic Nails” in 2002 he worked in the industry at ThyssenKrupp Automotive, first in Munich to develop intelligent chassis control systems and then in Liechtenstein to develop Electric Power Steering Systems (EPS). There he was responsible for the development and industrialization of the electrical, electronic and software components. After the first system went into mass production for the X3 BMW in 2010, he received the call from the Technical University of Applied Sciences Augsburg as professor in electrical engineering, responsible for Automatic Control and Dynamic Systems. Since then, he is also Faculty Advisor of the local Formula Student Team and leader of the research group DriverlessMobility. Since 2023 he is scientific director of the Technology Transfer Center Landsberg a. Lech, responsible for the Autonomous Systems.

PPENDIX
CORRECTNESS OF THE ENUMERATION ALGORITHM

Lemma 1 (Backtracking criterion for C_{seg} -constraint). The segment angle constraint C_{seg} can no longer be satisfied by adding more points to either boundary once it is violated.

Proof. Given the two boundary polygonal chains L and R represented by the path pair P and defined by the point sequence $(l_1, \dots, l_n)$ and $(r_1, \dots, r_m)$ respectively, the constraint $C_{seg}(P)$ is defined as follows:

$$C_{seg}(P) = \bigwedge_{i=1}^{N-2} |(\overline{l_i l_{i+1}}, \overline{l_{i+1} l_{i+2}})| < \phi_{max} \quad (15)$$

$$\wedge \bigwedge_{i=1}^{M-2} |(\overline{r_i r_{i+1}}, \overline{r_{i+1} r_{i+2}})| < \phi_{max}$$

By adding more vertices, more line segments are added and thus more consecutive angles are computed and therefore more boolean variables are added over which the conjunction is evaluated. Since a conjunction that is false cannot become true by adding more boolean variables to it, the C_{seg} constraint can no longer be satisfied by adding more points to either boundary once it is violated. $\square$

Lemma 2 (Backtracking criterion C_{poly} -constraint). The constraint C_{poly} requiring the driving lane polygon to be simple cannot be satisfied anymore by extending the lane except if it is violated by the line segment between the last points of the two boundaries intersecting any other line segment.

Proof. The driving lane polygon D is defined by the sequence of left boundary points concatenated with the right boundary points in reversed order: $D = (l_1, \dots, l_n, r_m, \dots, r_1)$. polygon is simple if no line segment of its boundary intersects any other (non-adjacent) line segment.

The line segment $\overline{l_n r_m}$ between the last points of the two boundaries changes every time the lane is extended (new points are added), since new points become the last ones. If therefore $\overline{l_n r_m}$ intersects any other segment, this intersection may be undone. If other edges of the polygon do not change as the lane is extended, therefore if any two segments intersect and none of them is $\overline{l_n r_m}$, this intersection cannot be undone and C_{poly} cannot be satisfied anymore. $\square$

Lemma 3 (Backtracking criterion C_{width} -constraint). If C_{width} is violated, it may be satisfied by extending the lane depending on the following three cases:

- 1) If it is violated by a fixed matching line being too long or too short, it cannot be satisfied anymore
- 2) If it is violated by a mutable matching line being too short, it cannot be satisfied anymore
- 3) If it is violated by a mutable matching line being too long, it may be satisfied by extending the boundaries

Proof. To show the first case, we observe that the online algorithm may only add more matching points to set of fixed matching points (see Alg. 3, line 8). Therefore, only more boolean variables are added to constraint Eq. 13 and since a conjunction that is false cannot become true by adding more

boolean variables to it, the constraint cannot be satisfied anymore once it is violated.

For the remaining two cases, observe that we can rewrite constraint 13 as:

$$C_{width}(P) = \min\{w_i \mid w_i \in W(L, R)\} > w_{min} \quad (16)$$

$$\wedge \max\{w_i \mid w_i \in W(L, R)\} < w_{max} \quad (17)$$

If the constraint violation is due to a matching line which belongs to the mutable set, it may be recomputed, we thus need to consider the computation of matching points (Eq. 9) which is based on nearest neighbor search. Consider the general case of a nearest neighbor search between the query and target sets $Q \subset \mathbb{R}^2$ and $T \subset \mathbb{R}^2$:

$$NN(Q, T) := \{\min_{t \in T} \{\|q_i - t_i\|_2\} \mid q_i \in Q\} \quad (18)$$

The following inequalities hold:

$$\max NN(Q, T) \leq \max NN(Q', T), Q \subset Q' \subset \mathbb{R}^2 \quad (19)$$

$$\min NN(Q, T) \geq \min NN(Q', T), Q \subset Q' \subset \mathbb{R}^2 \quad (20)$$

$$\max NN(Q, T) \geq \max NN(Q, T'), T \subset T' \subset \mathbb{R}^2 \quad (21)$$

$$\min NN(Q, T) \geq \min NN(Q, T'), T \subset T' \subset \mathbb{R}^2 \quad (22)$$

As the boundaries are extended, the query as well as the target domains of the NN-search becomes larger (Eq. 9 and 11). If the C_{width} -constraint is therefore violated by a matching line that belongs to the mutable set being too short (Eq. 16), from equations 20 and 22 it follows that it cannot become longer and therefore the constraint cannot be satisfied anymore by extending the boundaries, concluding the second case.

If however the C_{width} -constraint is violated by a mutable matching line that is too long (Eq. 17), from equation 21 it follows that it may become shorter by extending the boundaries and therefore the constraint may be satisfied, concluding the third case. $\square$

Theorem 4 (Backtracking does not exclude valid solutions 2). Backtracking in Algorithm 2 does not exclude candidates which would satisfy constraints.

Proof. Algorithm 2 backtracks in line 22, i.e. it does not extend the boundaries further. By lemmata 1, 2 and 3, backtracking does not exclude candidates which would satisfy the constraints and thus the enumeration is complete given a high enough iteration limit. $\square$